TAKE-HOME EXERCISE 3

Full-stack coding

Build an AI solution end to end, then walk us through it. This document contains everything you need — the pre-read, the exercise brief, and the submission criteria.

PART ONE	PART TWO	PART THREE
Pre-read	Exercise brief	Final output

PART ONE

Pre-read

Read this in advance of starting the take-home so you can prepare your environment and get access to LLM API keys before the coding exercise.

You will complete an async technical exercise that simulates early-stage, forward-deployed engineering: shipping a thin end-to-end slice with modern AI tooling. This is not a trivia test and not a no-AI test — you may use any coding agents and LLMs.

Time expectation	Roughly 2–3 hours	Going over	Fine — keep recording and tell us when you submit

Tools to have ready

Loom and dictation software

You should have received a separate candidate packet covering Loom and dictation setup. Go through it first so you can record and communicate easily with agents.

Coding agent — bring your own

Use whatever you already like: Cursor, Claude Code, and so on. If you do not have one, set one up beforehand.

LLM access

We will provide a baseline OpenAI API key for the exercise. If you prefer your own provider — OpenRouter or similar — that is fine.

Dev environment

You will build from scratch using a recommended stack. Have Git, Node and/or Python depending on your stack, and optionally Docker for the bonus.

What we evaluate

WE DO EVALUATE

- Can you ship a coherent end-to-end slice
- Product sense and UX empathy — basic states matter
- Judgment and prioritization
- Verification mindset — how you know it works
- How you use agents to move through unfamiliar areas

WE DO NOT EVALUATE

- Pixel-perfect UI polish
- Using a specific framework or tool
- Writing everything by hand

RECORDING REQUIREMENT

Record your whole build — screen plus audio, via Loom. Do not stress about being polished; we are looking at how you think and work.

PART TWO

Exercise brief

Build a small internal tool that helps triage inbound project requests quickly and consistently. You will implement a thin horizontal slice end to end: frontend, backend API, persistence, a small AI feature, basic UX states, and reproducibility.

Expect roughly 2–3 hours. If you want to keep hacking beyond that, go for it — just keep recording the whole time and mention it in your submission.

Choose a stack

Pick one of these. If you already have a preferred stack, choose the closest match and ship quickly.

OPTION A

Next.js + SQLite — recommended default

- Next.js App Router
- API via Route Handlers
- SQLite with Prisma (recommended) or Drizzle

OPTION B

FastAPI + React/Vite + SQLite

- FastAPI backend (Python)
- React/Vite frontend
- SQLite with SQLAlchemy or SQLModel

OPTION C

Node + React/Vite + SQLite

- Express or Fastify backend
- React/Vite frontend
- SQLite with Prisma (recommended) or Knex

ANYTHING ELSE

Only if it is truly your home base — Rails, Django, Next.js + Postgres, and the like. Email us first with your plan so we can confirm it fits the goals and the timebox.

Core requirements

1 Intake creation and persistence

An intake should include a title, description, budget range, timeline, industry (strings are fine for all three), and a `created_at` timestamp. Persist to a local database — SQLite recommended.

2 Frontend views

- Intake list view
- Intake detail view
- Create intake flow (form)

3 AI feature — required

On intake creation, call an LLM to generate a 2–3 sentence summary, three tags, and a risk checklist as bullets. Store those outputs and show them in the UI. Freeform text is fine; structured output against a JSON schema is a bonus, not a requirement.

4 UX states — we care about this

- Loading state
- Empty state
- Error state — at least one clear, user-visible error path

We care much more about clarity and empathy than visual polish.

Bonus, optional

Do any of these as you see fit — or go back to the core features and add some depth. We would love to see you show off.

- Dockerize it — Dockerfile or compose

- TypeScript (recommended, not required)
- Background job or async processing for the AI call
- Tests
- Basic dashboard (counts by tag or status) or CSV export
- Better AI reliability — retries, fallback behavior, guardrails

WHAT TO PRODUCE

Five components, summarized here for convenience. See the Final Output document for complete details.

- Full build recording — screen plus audio via Loom
- Walkthrough at the end of your recording
- Working app, delivered as a .zip
- Repo instructions (README)
- A short decision log (DECISIONS.md)

PART THREE

Final output

The final output we are looking for in your submission. Read this fully, and feel free to reach out with any questions before starting.

REQUIRED ARTIFACTS — PLEASE SUBMIT ALL FIVE

1 Loom recording

- Loom link
- Total time spent (approximate)
- If you went over 3 hours, note it here — totally fine

Record your entire build, screen plus audio. At the end, include a 10–20 minute wrap-up (below).

2 Walkthrough wrap-up

Wrap up the recording by walking us through:

A · PRODUCT AND UX

- The end-to-end flow: create intake → list → detail
- UX states: loading, empty, error

B · SYSTEM OVERVIEW

- How the frontend talks to the backend
- Data model and persistence
- Where AI is called and how results are stored

C · RELIABILITY AND VERIFICATION

- What you did to ensure correctness
- Failure modes you considered — LLM errors, bad input

D · TRADEOFFS AND NEXT STEPS

- What you intentionally did not do
- What you would do with one more day

3 Code as a zip of a local git repo

Please `git init` locally and commit as you build. Submit a zip of the repo directory, including the `.git` folder.

Recommended file name: `<FirstName>_<LastName>_Tribe_IntakeTriage.zip`

4 README.md

A · HOW TO RUN

- Prerequisites — Node, Python, etc.
- Install steps
- Env vars needed (LLM key)
- Commands to run backend and frontend
- URL to open locally

B · WHAT YOU BUILT

- Brief product overview, 1–3 bullets
- Implemented flow: create intake → list → detail
- AI feature behavior
- UX states included

C · VERIFICATION

- How you convinced yourself it works
- Tests, if any, and how to run them
- If no tests: a brief manual verification checklist

5 DECISIONS.md

- Plan — 2–4 bullets
- Key decisions and why
- How you verified it works
- Tradeoffs — what you did not do, and why
- What you learned, or would do differently next time
- If you had one more day: next steps

Feel free to dictate this and paste it in — short and concrete beats polished.

FINAL NOTE

We are explicitly not looking for perfection — we are looking for how you ship, prioritize, and validate.